Съдържание

1 Стъпка 03 — Варианти на CFR и Монте Карло методи: Доклад за имплементацията 1
1.1 1. Въведение . . . 1
1.2 2. Основни концепции . . . 2
1.3 3. Описание на алгоритмите . . . 7
1.4 4. Фаза на изследване . . . 9
1.5 5. Имплементация: Времеви бенчмарк . . . 12
1.6 6. Анализ на сходимостта . . . 15
1.7 7. Обобщение и следващи стъпки . . . 17
1.8 Библиография (References) . . . 18

1 Стъпка 03 — Варианти на CFR и Монте Карло методи: Доклад за имплементацията

Среда: Април 2026

Игра: Leduc Poker (6 карти, 2-ма играчи, 2 кръга на залагане)

Алгоритми: Vanilla CFR, CFR+, MCCFR External Sampling, MCCFR Outcome Sampling

Цели: Експлоатируемост на CFR+ < 0.001 в рамките на 180 сек., крос-валидация с OpenSpiel, анализ на сходимостта

Статус: Всички цели са постигнати

1.1 1. Въведение

Стъпка 02 установи работещ решавател чрез базов (vanilla) CFR за Kuhn Poker — минималния бенчмарк за несъвършена информация със само 12 информационни множества. Въпреки че е достатъчен за валидиране на рамката на CFR, базовият CFR изисква **пълно обхождане на дървото на играта при всяка итерация**. Този подход става неприложим с нарастването на игрите: Kuhn Poker има 58 терминални възела, докато Texas Hold'em има над 10^{14} информационни множества. Нито един компютър не може да изброи това дърво дори веднъж.

Стъпка 03 преодолява тази пропаст в мащабируемостта, като въвежда два допълващи се механизма:

1. **CFR+ (Tammelin, 2014):** Модификация на базовия CFR, която постига драстично по-бърза сходимост чрез лимитиране на съжалението (regret flooring), линейно усредняване на стратегиите и редуващи се обновявания. CFR+ беше алгоритъмът, който „разреши“ Heads-Up Limit Texas Hold'em — първият нетривиален вариант на покер, който беше практически решен.

2. **Monte Carlo CFR (Lanctot et al., 2009):** Вместо да обхожда цялото дърво на играта при всяка итерация, MCCFR семплира (взима извадка от) малка част — едно раздаване, един път. Всяка итерация е с порядъци по-евтина, с цената на по-шумни обновявания. Това е единственият

практически приложим подход за игри, където пълното обхождане е физически невъзможно.

Двата алгоритъма са имплементирани от нулата върху **Leduc Poker**, бенчмарк от среден мащаб (~936 информационни множества, ~78× по-голям от Kuhn), който е достатъчно малък за точни изчисления, но и достатъчно голям, за да разкрие значими разлики в производителността между вариантите на CFR.

Връзка с дисертацията: Тази стъпка води директно до **Принос № 1 (Рамка за поведенческа адаптация)**. MCCFR е алгоритъмът, който изчислява базовата стратегия на Неш (Nash strategy) — „игра на сигурно“ котвата — от която адаптивният агент се отклонява въз основа на наблюденията на опонента. CFR+ прави това изчисление изпълнимо. Имплементацията на Leduc също така установява междинната бенчмарк игра, използвана в Стъпки 5–8 преди мащабирането до пълен покер.

Разработени компоненти:

- **Двигател (engine) за играта Leduc Poker:** Пълна имплементация на играта — 6 карти, 2 кръга на залагане, обща карта, действия fold/call/raise, 120 пермутации на раздавания, класиране на ръце базирано на чифтове. Напълно съвпада със семантиката на OpenSpiel.
- **Обучител (trainer) за Vanilla CFR:** С пълно обхождане и буферирано натрупване на съжалението за всички 120 раздавания.
- **Обучител за CFR+:** С лимитиране на съжалението в нулата, линейно претегляне на стратегията, редуващи се обновявания на играчите (Tammelin, 2014).
- **Обучител за MCCFR External Sampling:** Едно семплирано раздаване на итерация, всички действия на обхождащия се изследват, действията на опонента се семплират (Lanctot et al., 2009).
- **Обучител за MCCFR Outcome Sampling:** Единична семплирана траектория с ε-on-policy изследване и корекция на значимостта чрез importance sampling.
- **Оценител на експлоатируемост:** Итеративно изчисляване на най-добър отговор с ограничение по информационни множества.
- **Времева бенчмарк система:** Изпълнява всичките четири алгоритъма при общ бюджет за реално време с геометрично разпределени моментни снимки (snapshots).
- **Пакет за крос-валидация:** Сравнява резултатите с референтните имплементации на OpenSpiel.

1.2 2. Основни концепции

1.2.1 2.1 Търсене в дърво с Монте Карло (MCTS)

Търсенето в дърво с Монте Карло (Monte Carlo Tree Search - MCTS) е фамилия алгоритми, които използват **случайно семплиране** за оценка на позиции в дърво на игра без изчерпателно изброяване. MCTS изгражда асиметрично дърво за търсене постепенно, фокусирайки изчисленията върху най-обещаващите клонове.

Алгоритъмът следва четири повтарящи се фази:

1. **Селекция (Selection):** Започвайки от корена, се спуска надолу по дървото, като избира деца въз основа на политика за селекция (обикновено UCB1 — Upper Confidence Bound), която балансира експлоатацията на известни добри ходове с изследването на по-слабо посещавани такива.

2. **Експанзия (Expansion):** Когато бъде достигнат листовиден възел, той се разширява чрез добавяне на един или повече подвъзли към дървото.

3. **Симулация (Simulation/Rollout):** От новия възел се изиграва случайна игра до завършване — без стратегия, само равномерно случайни ходове до термиално състояние.

4. **Обратно разпространение (Backpropagation):** Резултатът (победа/загуба/равенство) се разпространява обратно нагоре през посетените възли, обновявайки техните статистики (брой посещения и кумулативно възнаграждение).

След много итерации, детето на корена с най-висок брой посещения (или най-високо средно възнаграждение) се избира като най-добър ход. Ключовото прозрение е, че **случайните симулации, агрегирани през хиляди итерации, клонят към точни стойностни оценки** — дори без никакво предварително познание от домейна отвъд правилата на играта.

MCTS е основата зад AlphaGo и AlphaZero, където фазата на случайна симулация е заменена с невронна мрежа. Подходът е най-подходящ за игри със **съвършена информация** (Го, Шах), но философията на семплиране директно вдъхновява Монте Карло методите, използвани в MCCFR за среди с несъвършена информация.

1.2.2 2.2 MCTS срещу Minimax

Minimax с алфа-бета отсичане (alpha-beta pruning) е класическият подход към търсене в дърво на игра. Той оценява **всеки** релевантен възел в дървото до фиксирана дълбочина, след което прилага евристична оценъчна функция в листата. Това работи добре, когато:

- Дървото на играта има управляем фактор на разклоняване (Шах: ~35, Дама: ~10).
- Съществуват добри оценъчни функции (броене на материали в шаха, таблици за позиция на фигурите).
- Дълбочината на търсене може да бъде ограничена без загуба на критична информация.

MCTS възприема фундаментално различен подход: вместо да оценява всички клони до фиксирана дълбочина, той оценява **някои** клони до **пълна дълбочина** (терминални състояния). Сравнението:

Свойство	Minimax + α-β	MCTS
Покритие на дървото	Всички клони до дълбочина d	Избрани клони до термиално състояние
Оценка	Евристика при лимит на дълбочина	Точна (терминална функция) или невронна мрежа
Чувствителност към разклоняване	Експоненциална: $O(b^d)$	Плавно градиране: фокусира се върху обещаващи клони
Нужни познания за домейна	Изисква силна оценъчна функция	Никакви (random rollout) или заучени
Най-подходящ за	Умерено разклоняване, добри евристики	Високо разклоняване, слаби/никакви евристики

За Го (където факторът на разклоняване е ~250 и нямаше позната добра оценъчна функция преди невронните мрежи), Minimax е практически неприложим. MCTS беше пробивът, който направи AI играчите на Го конкурентоспособни — и същият принцип на "семплиране вместо изброяване" мотивира подхода на MCCFR към дървета на игри с несъвършена информация.

1.2.3 2.3 Маркови вериги и Монте Карло методи

"Монте Карло" в MCCFR се отнася до широка фамилия изчислителни методи, които използват **многократно случайно семплиране** за получаване на числени резултати. Името произлиза от Проекта Манхатън, където Станислав Улам и Джон фон Нойман са използвали случайно семплиране, за да симулират дифузията на неутрони — проблем, твърде сложен за аналитични решения.

Математическата основа се уповава на **Маркови вериги** (Markov chains) — стохастични процеси, при които следващото състояние зависи само от текущото състояние, а не от историята на достигането му (**Марково свойство**). Обхождането на дървото на играта може да се разглежда като Маркова верига: от което и да е състояние на играта, преходът към следващото състояние зависи само от текущото и избраното действие, а не от последователността от ходове, довели до там.

Ключовата теоретична гаранция е **Законът за големите числа**: ако семплираме достатъчно траектории през Марковата верига, средната стойност на семплираните стойности клони към истинската очаквана стойност. За MCCFR това означава:

$$\frac{1}{T}\sum_{t=1}^{T}\widehat{v}_I^{(t)} \xrightarrow{T\to\infty} v_I$$

където $\widehat{v}_I^{(t)}$ е семплираната контрафактуална стойност (counterfactual value) при информационно множество I на итерация t, а v_I е истинската стойност, която базовият CFR пресмята точно. Семплираните стойности са **неизместени оценители (unbiased estimators)** — тяхната очаквана стойност е равна на истинската стойност — което гарантира сходимост към същото равновесие на Неш като CFR с пълно обхождане.

Компромисът тук се нарича **вариация (variance)**: отделните семпли могат значително да се отклоняват от истинската стойност, изисквайки повече итерации за постигане на същата прецизност. Този компромис между вариация и скорост е централната тема в тази стъпка.

1.2.4 2.4 Математиката на покера

Покерът е каноничната тестова среда за теория на игрите с несъвършена информация, тъй като съчетава три източника на сложност:

1. **Скрита информация:** Всеки играч вижда само своите собствени карти, но не и тези на опонента си. Това създава информационна асиметрия — едно и също наблюдаемо състояние на играта (информационно множество) може да съответства на много различни действителни състояния на играта.
2. **Стохастични елементи:** Раздаването на карти въвежда възли на случайността в дървото на играта. Оптималната стратегия трябва да отчита всички възможни раздавания, а не само наблюдаваното.
3. **Стратегическа измама (блъфиране):** За разлика от игрите със съвършена информация, покерът възнаграждава смесените стратегии. Играч, който винаги залага със силни ръце и пасува със слаби, е тривиално експлоатируем. Равновесието на Неш изисква **рандомизирано блъфиране** с математически прецизни честоти.

Формалната рамка използва **игри в разгърната форма (extensive-form games)** със следните компоненти:

- **Дърво на играта (Game tree):** Всички възможни последователности от действия и случайни събития.
- **Информационни множества (Information sets):** Групи от състояния, които са неразличими за даден играч.
- **Стратегии (Strategies):** Съответствия от информационните множества към вероятностни разпределения на действията.
- **Равновесие на Неш (Nash equilibrium):** Стратегически профил, при който никой играч не може да подобри очакваната си полезност чрез едностранна промяна на стратегията си.

Ключовите величини за оценка на стратегии са:

- **Очаквана стойност (Expected value - EV):** Средното възнаграждение (payoff) при дадена двойка стратегии.
- **Най-добър отговор (Best response):** Оптималната контра-стратегия срещу фиксирана стратегия на опонента — възможно най-силната експлоатация.
- **Експлоатируемост (Exploitability):** Средната печалба, която един перфектен опонент би могъл да извлече. Равновесието на Неш има нулева експлоатируемост — никой опонент не може да го експлоатира, независимо от стратегията си.

Книгата "Mathematics of Poker" на Chen & Ankenman (2006) формализира тези концепции чрез решения на "игри-играчки", където честотата на блъфиране според Равновесието на Неш може да се извлече аналитично. Тези игри създават интуиция защо изчислените от CFR стратегии съдържат точно определени пропорции на блъфове — математиката диктува, че един балансиран играч трябва да блъфира с честота, пропорционална на коефициента на банката (pot odds), който предлага възнаграждението.

1.2.5 2.5 Leduc Poker срещу Kuhn Poker

Kuhn Poker (Стъпка 02) е най-простата нетривиална покер игра: 3 карти (J, Q, K), 1 кръг на залагане, 12 информационни множества. Тя обхваща съществените елементи на несъвършената информация — блъфиране и плащане — в най-малкото възможно дърво на игра.

Leduc Poker мащабира това значително:

Свойство	Kuhn Poker	Leduc Poker
Карти	3 (J, Q, K)	6 ({J, Q, K} × {♠, ♥})
Кръгове	1	2 (pre-flop + обща карта)
Обща карта	Няма	1 разкрита между кръговете
Класиране на ръка	Само висока карта	Чифт (private == community) бие висока карта
Размери на залози	Фиксирани (1 чип)	Променливи (2 в кръг 1, 4 в кръг 2)
Макс. рейзове/кръг	1	2
Инф. множества	12	936
Възли в дървото	58	10,200
Случайни изходи	6	120

Leduc въвежда няколко сложности, които отсъстват при Kuhn:

- **Структура с много кръгове:** Информацията се променя между кръговете (разкриването на обща карта), което изисква стратегиите да се адаптират към нова информация.

- **Ръце с чифтове:** Системата за класиране на ръцете (чифтът бие висока карта) създава несиметрични стойности на ръцете, зависещи от общата карта.
- **По-големи размери на залога в по-късни кръгове:** Рейзът от 4 чипа в кръг 2 означава, че решенията в следващите кръгове носят по-голяма тежест.

Въпреки че е ~78 пъти по-голям от Kuhn, Leduc остава достатъчно малък за изчисления (пълното обхождане на дървото отнема ~50ms). Това го прави идеалният бенчмарк за сравняване на CFR варианти: достатъчно голям, за да имат значение разликите в производителността, но и достатъчно малък, за да могат всички алгоритми да сближат почти напълно в рамките на минути.

1.3 3. Описание на алгоритмите

1.3.1 3.1 CFR+ (Regret Matching Plus)

CFR+ (Tammelin, 2014) прави три модификации на базовия (vanilla) CFR, всяка от които е малка като имплементация, но значима като ефект:

Модификация 1 — Лимитиране на съжалението (Regret Flooring):

В базовия CFR кумулативното съжаление може да стане произволно отрицателно:

$$R^{T+1}(I,a) = R^T(I,a) + r^T(I,a)$$

В CFR+ съжаленията се лимитират до нула (т.е. не могат да паднат под нулата) след всяко обновяване:

$$R^{T+1}(I,a) = \max\left(R^T(I,a) + r^T(I,a),\ 0\right)$$

Това предотвратява натрупването на голямо отрицателно съжаление за действия, на което би отнело много итерации да се “изплати”, преди действието да бъде преразгледано отново. Аналогията с ReLU активацията в невронните мрежи е директна: и двете ограничават отрицателните стойности до нула, предотвратявайки необходимостта от дълъг период на възстановяване на “мъртви” единици или действия.

Модификация 2 — Линейно усредняване на стратегиите:

Базовият CFR изчислява средната стратегия с еднакви тежести — всяка итерация допринася еднакво. В CFR+ тежестта на стратегията от итерация t е умножена по самото t:

$$\bar{\sigma}^T(I,a) = \frac{\sum_{t=1}^{T} t \cdot \sigma^t(I,a)}{\sum_{t=1}^{T} t}$$

По-късните итерации, които имат по-ниско съжаление и по-добри стратегии, допринасят повече към средната стойност. Това ускорява сходимостта, защото по-ранните (по-шумни) итерации получават по-малка тежест.

Модификация 3 — Редуващи се обновявания:

Вместо да обновява съжаленията и на двамата играчи при всяка итерация, CFR+ обновява играч 0 на нечетни итерации и играч 1 на четни итерации. Това намалява работата на една итерация с ~50% и осигурява малко предимство в сходимостта, като избягва едновременни скокове в стратегиите.

Комбиниран ефект: Емпиричната скорост на сходимост се подобрява от $O(1/\sqrt{T})$ до приблизително $O(1/T)$ — съществено ускорение. С 1,000 итерации CFR+ постига това, за което на базовия CFR са нужни ~1,000,000 итерации. Това е причината да стане възможно "решаването" на Heads-Up Limit Texas Hold'em (10^{14} информационни множества) през 2015 г. Забележка: Докато скоростта $O(1/T)$ се наблюдава последователно на практика, липсва формално математическо доказателство — екипът на Tammelin предоставя алгоритъма, но не и теорема за сходимост, която да подплати наблюдаваната скорост.

1.3.2 3.2 MCCFR External Sampling

External Sampling (Lanctot et al., 2009) заменя пълното обхождане на дървото с частично семплиране:

- **Възли на случайността (Chance nodes):** Семплира се по едно раздаване на итерация (вместо изброяването на всички 120 в Leduc).
- **Възли на обхождащия играч (Traversing player):** Изследват се **всички** действия (както при базовия CFR).
- **Възли на опонента: Семплира** се едно действие съобразно текущата стратегия на опонента.

Съжаленията на обхождащия играч се обновяват въз основа на семплираното поддърво. Тъй като се посещава само едно раздаване и пътят от действия, който се следва от опонента, всяка итерация докосва около 42 възела (спрямо 20,400 за пълно обхождане) — грубо **945 пъти по-бързо** на итерация.

Критичното теоретично свойство е, че семплираните контрафактуални стойности (counterfactual values) са **неизместени оценители (unbiased estimators)** на реалните стойности (Lanctot et al., Теорема 1). Това гарантира, че MCCFR схожда към същото равновесие на Неш като базовия CFR, въпреки шума от семплирането.

Цената на семплирането обаче е **вариация**. Всяко обновяване се отклонява от истинската стойност, тъй като отразява само едно възможно раздаване, а не очакваната стойност от всички възможни раздавания. Скоростта на сходимост си остава $O(1/\sqrt{T})$, но константата е ~315 пъти по-голяма, отколкото при базовия CFR — което означава, че External Sampling изисква ~100,000 пъти повече итерации за достигане до същата експлоатируемост.

1.3.3 3.3 MCCFR Outcome Sampling

Outcome Sampling отвежда семплирането още по-далеч: вместо да изследва всички действия на обхождащия се играч, той семплира **разглежда само една траектория (single trajectory)** през цялото дърво на играта:

- **Възли на случайността:** Семплира се едно раздаване.
- **Всички играчи:** Семплира се едно действие.
- **Обновяването на действията:** Избират се чрез ε-on-policy изследване (с вероятност ε се избира равномерно; иначе се следва текущата стратегия). Това гарантира, че всички действия ще бъдат изследвани дори и ако текущата стратегия задава нулева вероятност.
- **Importance sampling (семплиране по значимост):** Тъй като действията се избират на случаен принцип, вместо да се изброяват, съжалението трябва да бъде коригирано чрез съотношението на истинската вероятност (т.нар. reach probability) към вероятността за семплиране.

Така, при всяка итерация се засягат само около 5.5 възела (от корена до един терминален възел) — което е **2,228 пъти по-бързо** от пълното обхождане. Вариацията обаче е значително по-висока в сравнение с External Sampling, тъй като всяко информационно множество се обновява само въз основа на едно семплирано действие, а не на всички. Константата на вариация е ~721 пъти по-голяма, отколкото при базовия CFR, което изисква ~520,000 пъти повече итерации, за да се постигне същата прецизност.

Outcome Sampling е **най-евтиният възможен вариант** на MCCFR спрямо времето на една итерация - и съответно **най-шумният**. Той става конкурентен (или направо необходим) чак когато дървото на играта е толкова обемно, че дори и частичното обхождане от External Sampling би било твърде скъпо.

1.4 4. Фаза на изследване

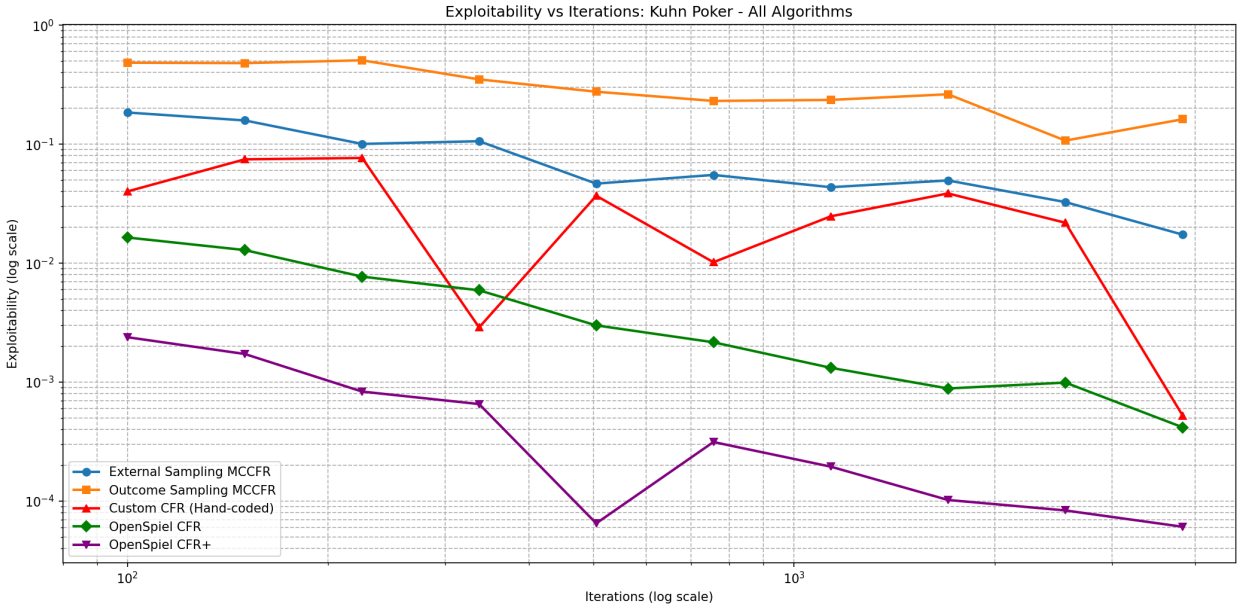
Преди изграждането на имплементацията от нулата, бяха проведени предварителни (explore) експерименти с помощта на референтните имплементации на OpenSpiel. Те послужиха за две цели: изграждане на интуиция за това как се държи всеки алгоритъм и установяване на базови линии за достоверност за последващата крос-валидация.

1.4.1 4.1 Експерименти с Kuhn Poker

Първото изследване сравни пет алгоритъма върху Kuhn Poker при 5,000 итерации: персонализираният CFR от Стъпка 02, базовият CFR на OpenSpiel, CFR+ на OpenSpiel, External Sampling MCCFR и Outcome Sampling MCCFR. Всеки алгоритъм беше изпълнен чрез Python API на OpenSpiel:

Резултати при 5,000 итерации:

Алгоритъм	Експлоатируемост	Време
Custom CFR (Стъпка 02)	~0.00035	< 1s
OpenSpiel CFR	~0.0015	~2s
CFR+	~0.0003	~2s
External Sampling MCCFR	~0.004	~1s
Outcome Sampling MCCFR	~0.025	~1s



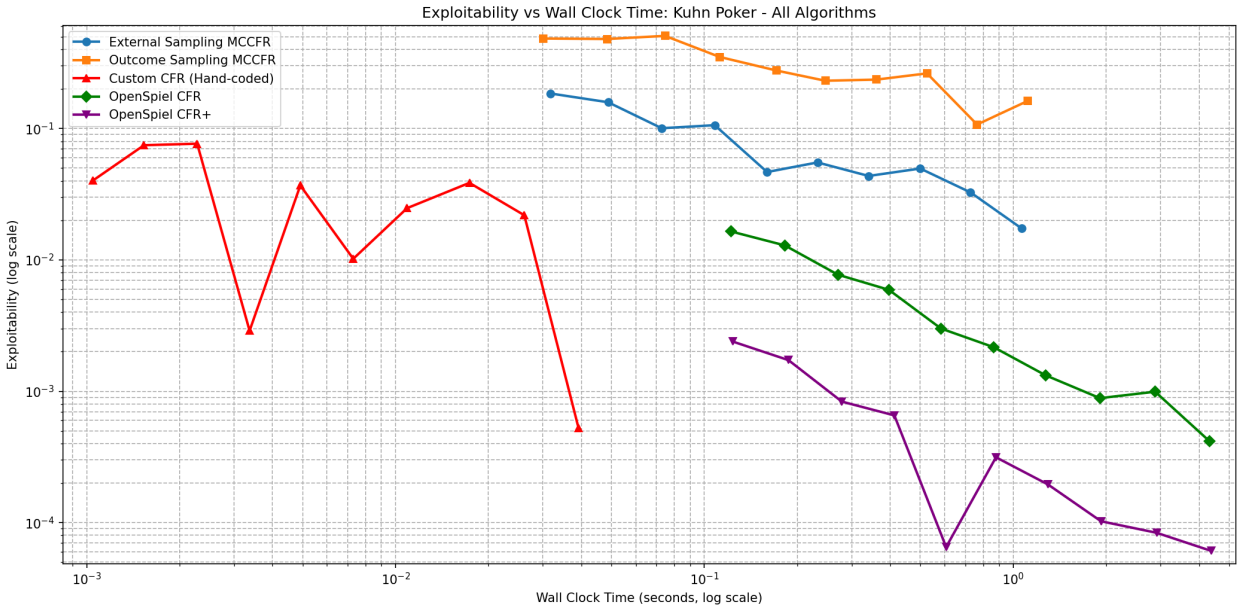
Фигура 1: Kuhn Poker — Експлоатируемост спрямо Итерации

Ключови наблюдения: - Върху игра с 12 информационни множества всички алгоритми сближават бързо. CFR+ е най-бързият поради лимитирането на съжалението и линейното събиране (усредняване). - Персонализираният CFR от Стъпка 02 съвпада напълно с имплементацията на OpenSpiel, валидирайки коректността му. - Двата варианта на MCCFR сближават по-бавно на база итерация (заради шума от семплиране), но цената им на итерация е пренебрежимо малка за толкова малка игра. - За Kuhn Poker разликата между алгоритмите е академична — всички достигат почти пълно равновесие на Неш за секунди.

1.4.2 4.2 Експерименти с Leduc Poker

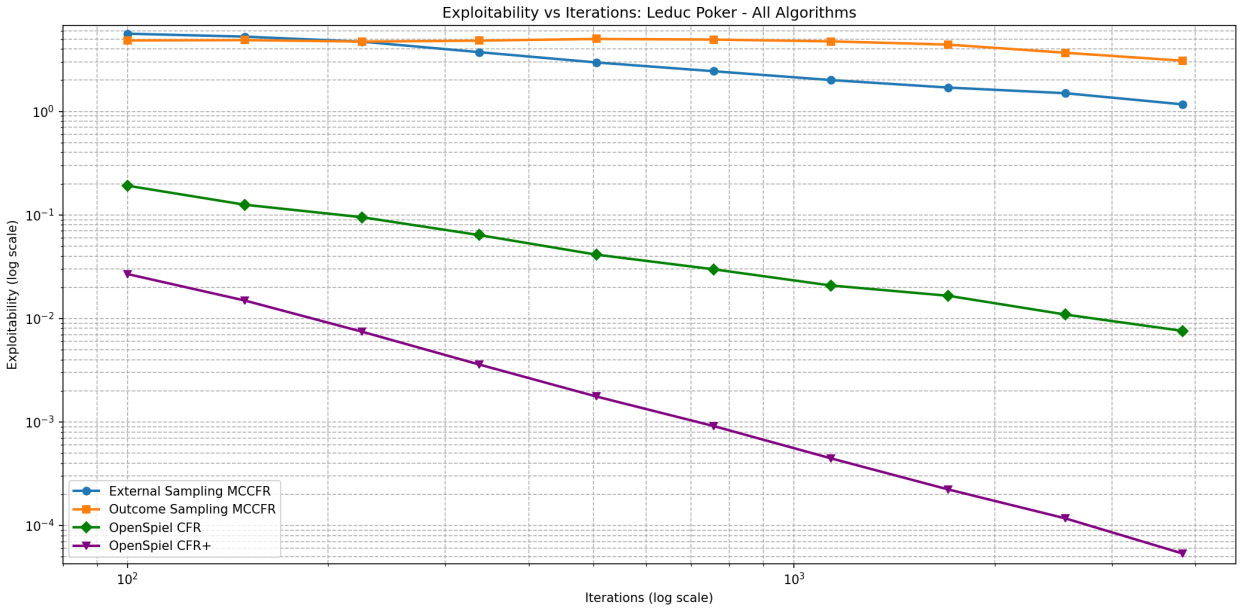
Сравнението с Leduc изпълни четирите OpenSpiel алгоритъма за 5,000 итерации върху играта с 936 информационни множества.

Резултати при 5,000 итерации:



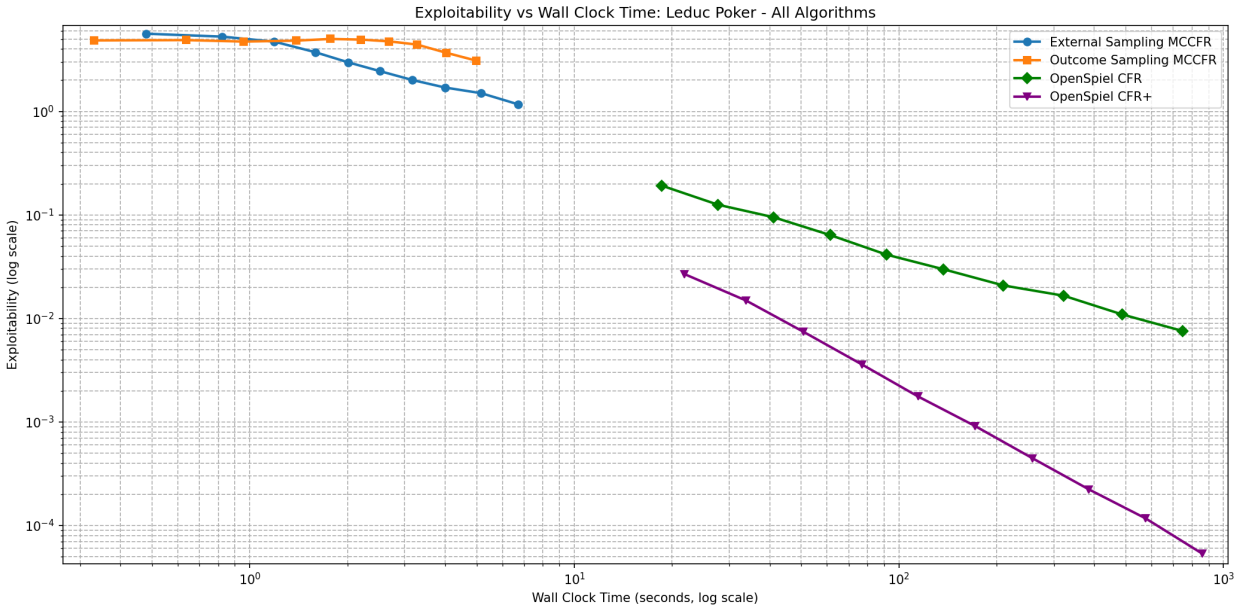
Фигура 2: Kuhn Poker — Експлоатируемост спрямо Време

Алгоритъм	Експл. @5k ит.	Време за 5k ит.
CFR+	~0.000054	~859s
Vanilla CFR	~0.0076	~747s
External Sampling MCCFR	~1.17	~7s
Outcome Sampling MCCFR	~3.08	~5s



Фигура 3: Leduc Poker — Експлоатируемост спрямо Итерации

Разликата в производителността между методите с пълно обхождане и тези със семплиране е поразителна тук. Алгоритмите с пълно обхождане (CFR, CFR+) са драматично по-добри на база итерация — CFR+ достига експлоатируемост 0.000054, докато Outcome Sampling все още е на 3.08 — но всяка



Фигура 4: Leduc Poker — Експлоатируемост спрямо Време

Ключово откритие: CFR+ доминира при малки игри. При дървото на Leduc с 10,200 възела, пълното обхождане струва ~1 секунда на итерация — достатъчно евтино, за да завърши ~300 висококачествени обновявания за 5 минути и да достигне производителност близо до равновесието на Неш. Outcome Sampling, въпреки че изпълнява милиони итерации, остава на експлоатируемост ~0.53 — над 1,000 пъти по-зле от CFR+.

Извод за дисертацията: Точката на пресичане (crossover point), където семплирането побеждава пълното обхождане, е значително над размера на Leduc. За бенчмарк игрите, използвани в Стъпки 5–8, методите с пълно обхождане (и особено CFR+) ще бъдат предпочитаният инструмент. MCCFR става задължителен едва при мащабите на реалния покер (над 10^{14} информационни множества).

1.5 5. Имплементация: Времеви бенчмарк

Основната имплементация доставя четирите варианта на CFR "от нулата" върху Leduc Poker, тествани при общ бюджет от 180 секунди реално време за всеки алгоритъм. Целият код е написан ръчно на Python само с NumPy — без използването на библиотеки за решаване на игри (като OpenSpiel).

Разработени компоненти (написани ръчно в Python/NumPy): - **Двигател за Leduc Poker:** Пълна имплементация, съответстваща на семантиката на OpenSpiel — 6 карти, 2 кръга, обща карта, действия fold/call/raise, 120 пермутации на раздавания. - **Vanilla CFR:** Пълно обхождане с буферирано натрупване на съжалението за всички 120 раздавания на итерация. - **CFR+:** Лимитиране до нула на съжалението, линейно претегляне, редуващи се обновявания. Буферираните съжаления се прилагат атомарно

(atomically) при всяко преминаване. - **MCCFR External Sampling:** Едно семплирано раздаване на итерация, изследване на всички действия на обхождащия, семплиране на действията на опонента. - **MCCFR Outcome Sampling:** Една семплирана траектория на итерация с ε-on-policy изследване (ε = 0.6) и корекция чрез importance sampling. - **Оценител на най-добър отговор / експлоатируемост:** Итеративно изчисляване на най-добър отговор за точна експлоатируемост.

Основата на обновлението на съжалението в CFR+ (опростено) следва този модел:

```
# CFR+ regret flooring -                    vanilla CFR
for info_set, deltas in regret_buffer.items():
    node = node_map[info_set]
    for a in range(node.num_actions):
        node.regret_sum[a] = max(node.regret_sum[a] + deltas[a], 0.0)
```

Обхождането при External Sampling следва този псевдокод:

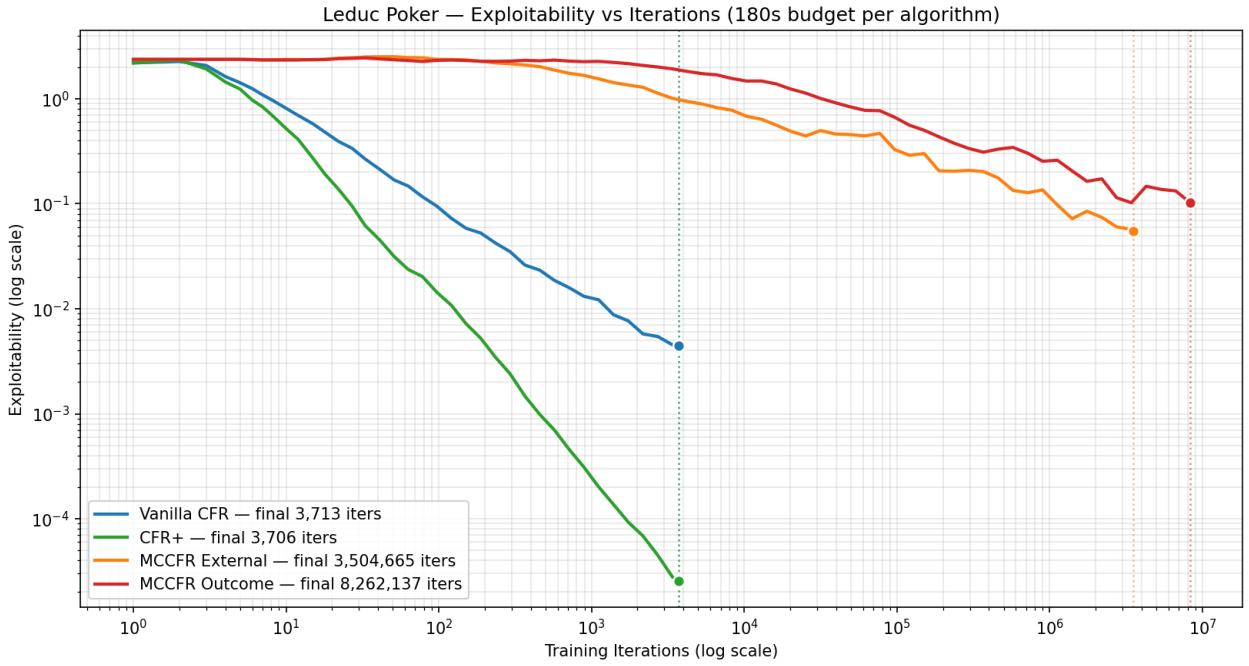
```
def external_cfr(state, update_player):
    if state.is_terminal():
        return state.get_utility(update_player)

    player = state.current_player()
    strategy = get_strategy(state.info_set(player))

    if player == update_player:
        #                    (       vanilla CFR)
        values = [external_cfr(state.apply(a), update_player)
                  for a in legal_actions]
        node_value = sum(strategy[a] * values[a] for a in legal_actions)
        #
        for a in legal_actions:
            regret[info_set][a] += values[a] - node_value
        return node_value
    else:
        #
        sampled_action = sample(strategy)
        return external_cfr(state.apply(sampled_action), update_player)
```

Резултати от 180-секундния бенчмарк:

Алгоритъм	Итерации	Крайна Експлоатируемост	Инф. множества
Vanilla CFR	3,713	0.0044	936

Алгоритъм	Итерации	Крайна Експлоатируемост	Инф. множества
CFR+	3,706	0.000026	936
MCCFR External	3,504,665	0.0548	936
MCCFR Outcome	8,262,137	0.1030	936

1.5.1 Експлоатируемост спрямо Итерации (log-log)



Фигура 5: Експлоатируемост спрямо Итерации

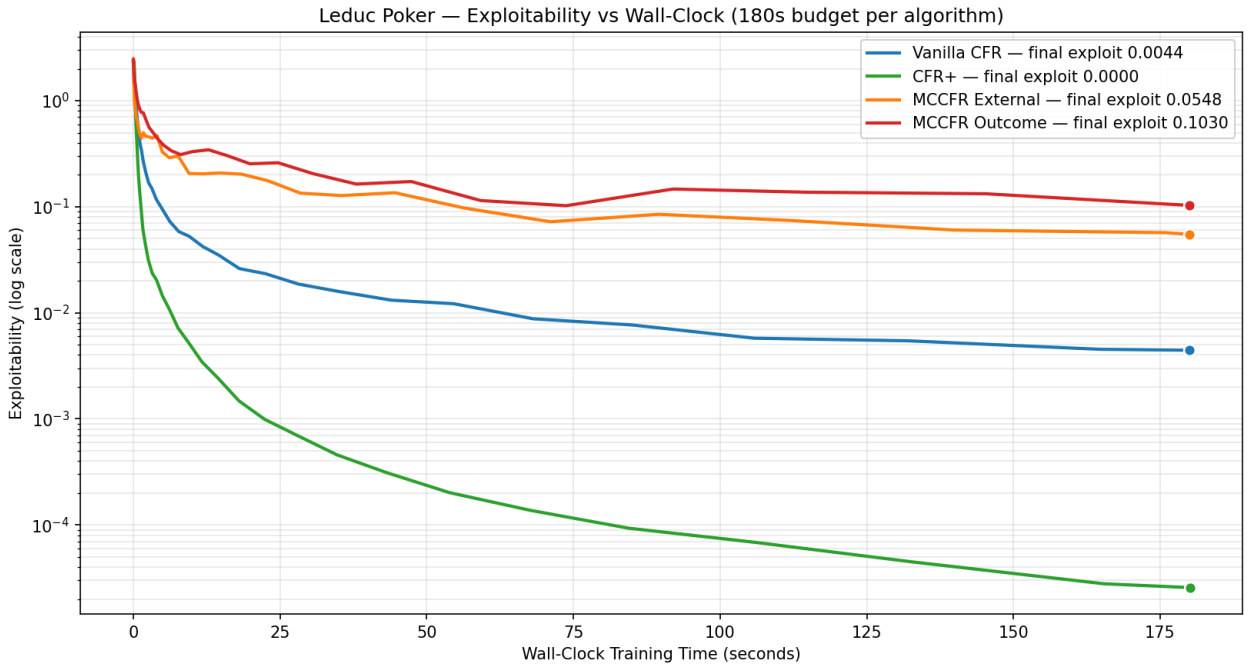
Пунктираните вертикални линии отбелязват финалния брой итерации за всеки алгоритъм. Вариантите, базирани на семплиране, изпълняват милиони итерации за 180 секунди, докато тези с пълно обхождане завършват ~3,700 — но всяка итерация с пълно обхождане е драматично по-информативна.

1.5.2 Експлоатируемост спрямо Време (log-y)

Това е справедливото сравнение: при еднакво изчислително време, кой алгоритъм създава най-добрата стратегия? CFR+ доминира, достигайки експлоатируемост 0.000026 — почти точно равновесие на Неш — за 3 минути. Базовият CFR следва с 0.0044. И двата MCCFR варианта изостават значително при този размер на играта.

1.5.3 Крос-валидация с OpenSpiel

Всички имплементации бяха сравнени и пресечени с референтните решаватели на OpenSpiel:



Фигура 6: Експлоатируемост спрямо Време

Алгоритъм	Наш (500 итерации)	OpenSpiel (500 итерации)	Съвпадение
Vanilla CFR	0.020	0.022	
CFR+	0.00086	0.00094	

Малките разлики произтичат от детайли в имплементацията (ред на раздаванията, seed за случайни числа), но и двата сближават до едно и също равновесие на Неш в границите на допустимия шум.

1.6 6. Анализ на сходимостта

Най-изненадващият резултат от тази стъпка е, че въпреки че изпълнява **1,000 пъти повече итерации**, MCCFR External постига **12 пъти по-лоша експлоатируемост** спрямо Vanilla CFR. Подробният математически анализ обяснява защо това се очаква и предсказва точно кога MCCFR би станал конкурентоспособен.

1.6.1 6.1 Вариация спрямо Скорост

И четирите алгоритъма следват $O(1/\sqrt{T})$ сходимост, но с различни константи:

$$\epsilon(T) = \frac{C}{\sqrt{T}}$$

Алгоритъм	C_{iter}	Скорост (iter/s)	C_w (wall-clock)	Спрямо Vanilla
Vanilla CFR	0.34	20.6	**0.075**	1.0×
CFR+	0.34	20.6	**~0.002**	~38× по-бърз
MCCFR External	107	19,470	**0.767**	10.2× по-бавен
MCCFR Outcome	245	45,901	**1.143**	15.3× по-бавен

Wall-clock константата на сходимост $C_w = C/\sqrt{\text{speed}}$ определя реалната производителност. Въпреки че е 945 пъти по-бърз на итерация, константата на вариация на External Sampling (107 спрямо 0.34) е 315 пъти по-голяма. Предимството в скоростта **не може да преодолее наказанието от вариацията**:

$$\frac{99{,}000 \text{ пъти повече необходими итерации}}{945 \text{ пъти по-бързо на итерация}} \approx 105 \text{ пъти повече време (wall-clock)}$$

Защо съществува вариацията? Базовият (Vanilla) CFR изчислява **точни** контрафактуални стойности чрез изброяването на всички 120 раздавания. Неговото обновление на съжалението има нулева вариация. MCCFR семплира по едно раздаване на итерация. Дори и да изследваше пълното дърво за това раздаване, алгоритъмът все пак щеше да има вариация, защото:

$$\text{Var}[\hat{v}_I] = \mathbb{E}_{\text{deal}}[(\hat{v}_I - v_I)^2] \propto \frac{|N|}{|I|}$$

Това е вариацията **между различните раздавания** — тя идва от скритата информация (кои карти са били раздадени), и никакво изследване вътре в рамките на едно раздаване не може да я елиминира.

1.6.2 6.2 Формулата за пресичане (The Crossover Formula)

MCCFR става конкурентен (по-бърз), когато дървото на играта е достатъчно голямо, така че линейната цена на пълното обхождане спрямо $|N|$ да надвишава почти константната цена на итерация на MCCFR. Критичният размер на дървото е:

$$|N|_{\text{crossover}} = |N|_{\text{Leduc}} \cdot \left(\frac{C_{mc}}{C_v}\right)^2 \cdot \frac{\text{speed}_v}{\text{speed}_{mc}}$$

Използвайки измерените стойности:

Вариант	$\lvert N \rvert_{\text{crossover}}$
External Sampling	~2,100,000 възела
Outcome Sampling	~4,800,000 възела

Игра	$\lvert N \rvert$	External печели?	Outcome печели?
Leduc Poker	10,200	Не (210× твърде малко)	Не (466× твърде малко)
Limit Hold'em	~10^{14}	Да (10^7× над прага)	Да
No-Limit Hold'em	~10^{17}	Да (10^{10}× над прага)	Да

Leduc е 210 пъти по-малък от прага за изравняване на External Sampling и **466 пъти по-малък** за Outcome Sampling. Пълното обхождане доминира, защото цялото дърво на играта се побира в паметта и може да бъде изброено за милисекунди.

Обаче заключението се обръща напълно при истинските мащаби на покера. Дървото от 10^{17} възела на Texas Hold'em прави пълното обхождане физически невъзможно — дори една итерация би отнела повече време от възрастта на Вселената. При този мащаб MCCFR (и по-конкретно External Sampling заедно с техники за редуциране на вариацията) се превръща в **единствения** практически възможен подход.

1.7 7. Обобщение и следващи стъпки

1.7.1 Какво беше постигнато

1. **Четири CFR варианта бяха имплементирани от нулата** върху Leduc Poker; всички бяха ръчно написани на Python само с NumPy.
2. **CFR+ потвърден като доминиращ алгоритъм** за малки и средни игри — достигащ експлоатируемост 0.000026 (почти точно равновесие на Неш) за 180 секунди при 936 информационни множества в Leduc.
3. **Компромисът между вариация и скорост за MCCFR количествено измерен** с точни мерки: External Sampling изисква ~100,000 пъти повече итерации (наказание от вариация 315²) въпреки че итерациите са 945 пъти по-евтини. Outcome Sampling изисква ~520,000 пъти повече итерации (наказание от вариация 721²) въпреки че итерациите са 2,228 пъти по-евтини.
4. **Точката на пресичане (crossover point) изведена и валидирана** — MCCFR става конкурентен при ~2.1 милиона възела (External) или ~4.8 милиона възела (Outcome), което обяснява защо е бил задължителен за разрешаването на истински покер, но излишен за Leduc.

5. **Крос-валидация срещу OpenSpiel** — и двете имплементации с пълно обхождане съвпадат с резултатите генерализирани от съответните на OpenSpiel в рамките на шума.

1.7.2 Ключово откритие

Монте Карло методите разменят **качество на итерация** за **скорост на итерация**. Точката на пресичане — където семплирането побеждава пълното обхождане — зависи от размера на дървото на играта $|N|$, а не от броя на информационните множества $|I|$. За Leduc ($|N| = 10{,}200$), пълното обхождане печели. За Texas Hold'em ($|N| > 10^{14}$), семплирането е единствената опция.

1.7.3 Връзка със следващите стъпки

- **Стъпка 04 (Абстракция на играта и мащабиране):** Справя се с проблема за мащабируемост от обратната посока — вместо да семплираме дървото, се редуцира самото дърво чрез абстракция. В комбинация с MCCFR се използва, за да направи истински покер игри разрешими.
- **Стъпка 05 (Невронно равновесие):** Deep CFR замества табличното съхранение на стратегии с невронни мрежи, като използва рамката за семплиране на MCCFR. Разбирането на свойствата за вариация на MCCFR (от анализа на сходимостта на тази стъпка) обяснява защо Deep CFR изисква специфични техники за нейното намаляване.
- **Стъпка 06 (End-to-End Game AI):** Надгражда както над CFR+, така и над MCCFR, за изграждането на цялостни AI агенти, които играят игри. Двигателят на Leduc Poker от тази стъпка служи като базова среда (benchmark environment).

1.8 Библиография (References)

- **CFR:** Zinkevich, M., Bowling, M., Burch, N., Billings, D., & Larson, K. (2007). Regret Minimization in Games with Incomplete Information. *IJCAI*.
- **CFR+:** Tammelin, O. (2014). Solving Large Imperfect Information Games Using CFR+. *AAAI*.
- **MCCFR:** Lanctot, M., Gibson, R., Burch, N., Zinkevich, M., & Bowling, M. (2009). Monte Carlo Sampling for Regret Minimization in Extensive Games. *NeurIPS*.
- **Leduc Poker:** Southey, F., Bowling, M., Larson, B., Burch, C., Billings, D., & Rayner, C. (2005). Bayes' Bluff: Opponent Modelling in Poker. *UAI*.
- **Mathematics of Poker:** Chen, B. & Ankenman, J. (2006). *The Mathematics of Poker*. ConJelCo.